Coding Conventions und Metriken

Carsten Gips (HSBI)

TL;DR
Code entsteht nicht zum Selbstzweck, er muss von anderen Menschen leicht verstanden und gewartet werden können: Entwickler verbringen einen wesentlichen Teil ihrer Zeit mit dem **Lesen** von (fremdem) Code.
Dabei helfen "Coding Conventions", die eine gewisse einheitliche äußerliche Erscheinung des Codes vorgeben (Namen, Einrückungen, …). Im Java-Umfeld ist der "Google Java Style" bzw. der recht ähnliche "AOSP Java Code Style for Contributors" häufig anzutreffen. Coding Conventions beinhalten typischerweise Regeln zu

- Schreibweisen und Layout
- Leerzeichen, Einrückung, Klammern
- Zeilenlänge, Umbrüche
- Kommentare

Die Beachtung von grundlegenden Programmierprinzipien hilft ebenso, die Lesbarkeit und Verständlichkeit zu verbessern.
Metriken sind Kennzahlen, die aus dem Code berechnet werden, und können zur Überwachung der Einhaltung von Coding Conventions und anderen Regeln genutzt werden. Nützliche Metriken sind dabei NCSS (*Non Commenting Source Statements*), McCabe (*Cyclomatic Complexity*), BEC (*Boolean Expression Complexity*) und DAC (*Class Data Abstraction Coupling*).
Für die Formatierung des Codes kann man die IDE nutzen, muss dort dann aber die Regeln detailliert manuell einstellen. Das Tool **Spotless** lässt sich dagegen in den Build-Prozess einbinden und kann die Konfiguration über ein vordefiniertes Regelset passend zum Google Java Style/AOSP automatisiert vornehmen.
Die Prüfung der Coding Conventions und Metriken kann durch das Tool **Checkstyle** erfolgen. Dieses kann beispielsweise als Plugin in der IDE oder direkt in den Build-Prozess eingebunden werden und wird mit Hilfe einer XML-Datei konfiguriert.
Um typische Anti-Pattern zu vermeiden, kann man den Code mit sogenannten *Lintern* prüfen. Ein Beispiel für die Java-Entwicklung ist **SpotBugs**, welches sich in den Build-Prozess einbinden lässt und über 400 typische problematische Muster im Code erkennt.
Für die Praktika in der Veranstaltung Programmiermethoden wird der Google Java Style oder AOSP genutzt. Für die passende Checkstyle-Konfiguration wird eine minimale checkstyle.xml bereitgestellt (vgl. Folie "Konfiguration für das PM-Praktikum").
Videos

- VL Coding Conventions
- Demo Formatter und Spotless
- Demo Checkstyle
- Demo Checkstyle: Konfiguration mit Eclipse-CS
- Demo SpotBugs

---

1 Coding Conventions: Richtlinien für einheitliches Aussehen von Code

=> Ziel: Andere Programmierer sollen Code schnell lesen können

- **Namen, Schreibweisen**: UpperCamelCase vs. lowerCamelCase vs. UPPER_SNAKE_CASE
- **Kommentare** (Ort, Form, Inhalt): Javadoc an allen `public` und `protected` Elementen

- **Einrückungen und Spaces vs. Tabs**: 4 Spaces
- **Zeilenlängen**: 100 Zeichen
- **Leerzeilen**: Leerzeilen für Gliederung
- **Klammern**: Auf selber Zeile wie Code

Beispiele: Sun Code Conventions, Google Java Style, AOSP Java Code Style for Contributors

2 Beispiel nach Google Java Style/AOSP formatiert

```java
package wuppie.deeplearning.strategy;

/**
 * Demonstriert den Einsatz von AOSP/Google Java Style ................
     Umbruch nach 100 Zeichen |
 */
public class MyWuppieStudi implements Comparable<MyWuppieStudi> {
    private static String lastName;
    private static MyWuppieStudi studi;

    private MyWuppieStudi() {}

    /** Erzeugt ein neues Exemplar der MyWuppieStudi-Spezies (max. 40 Zeilen)
        */
    public static MyWuppieStudi getMyWuppieStudi(String name) {
        if (studi == null) {
            studi = new MyWuppieStudi();
        }
        if (lastName == null) lastName = name;

        return studi;
    }

    @Override
    public int compareTo(MyWuppieStudi o) {
        return lastName.compareTo(lastName);
    }
}
```

Dieses Beispiel wurde nach Google Java Style/AOSP formatiert.

Die Zeilenlänge beträgt max. 100 Zeichen. Pro Methode werden max. 40 Zeilen genutzt. Zwischen Attributen, Methoden und Importen wird jeweils eine Leerzeile eingesetzt (zwischen den einzelnen Attributen *muss* aber keine Leerzeile genutzt werden). Zur logischen Gliederung können innerhalb von Methoden weitere Leerzeilen eingesetzt werden, aber immer nur eine.

Klassennamen sind UpperCamelCase, Attribute und Methoden und Parameter lowerCamelCase, Konstanten (im Beispiel nicht vorhanden) UPPER_SNAKE_CASE. Klassen sind Substantive, Methoden Verben.

Alle `public` und `protected` Elemente werden mit einem Javadoc-Kommentar versehen. Überschriebene Methoden müssen nicht mit Javadoc kommentiert werden, müssen aber mit `@Override` markiert werden.

Geschweifte Klammern starten immer auf der selben Codezeile. Wenn bei einem `if` nur ein Statement vorhanden ist und dieses auf die selbe Zeile passt, kann auf die umschließenden geschweiften Klammern ausnahmsweise verzichtet werden.

Es wird mit Leerzeichen eingerückt. Google Java Style arbeitet mit 2 Leerzeichen, während AOSP hier 4 Leerzeichen vorschreibt. Im Beispiel wurde nach AOSP eingerückt.

Darüber hinaus gibt es vielfältige weitere Regeln für das Aussehen des Codes. Lesen Sie dazu entsprechend auf Google Java Style und auch auf AOSP nach.

3 Formatieren Sie Ihren Code (mit der IDE)

Sie können den Code manuell formatieren, oder aber (sinnvollerweise) über Tools formatieren lassen. Hier einige Möglichkeiten:

- IDE: Code-Style einstellen und zum Formatieren nutzen
- google-java-format: `java -jar google-java-format.jar --replace *.java` (auch als IDE-Plugin)
- **Spotless** in Gradle:

```
plugins {
    id "java"
    id "com.diffplug.spotless" version "6.5.0"
}

spotless {
    java {
        // googleJavaFormat()
        googleJavaFormat().aosp()  // indent w/ 4 spaces
    }
}
```

  Prüfen mit `./gradlew spotlessCheck` (Teil von `./gradlew check`) und Formatieren mit `./gradlew spotlessApply`

3.1 Einstellungen der IDE's

- Eclipse:
  - `Project > Properties > Java Code Style > Formatter`: Coding-Style einstellen/einrichten
  - Code markieren, `Source > Format`
  - Komplettes Aufräumen: `Source > Clean Up` (Formatierung, Importe, Annotationen, ...) Kann auch so eingestellt werden, dass ein "Clean Up" immer beim Speichern ausgeführt wird!
- IntelliJ verfügt über ähnliche Fähigkeiten:
  - Einstellen über `Preferences > Editor > Code Style > Java`

- Formatieren mit `Code > Reformat Code` oder `Code > Reformat File`

Die Details kann/muss man einzeln einstellen. Für die "bekannten" Styles (Google Java Style) bringen die IDE's oft aber schon eine Gesamtkonfiguration mit.

Achtung: Zumindest in Eclipse gibt es mehrere Stellen, wo ein Code-Style eingestellt werden kann ("Clean Up", "Formatter", ...). Diese sollten dann jeweils auf den selben Style eingestellt werden, sonst gibt es unter Umständen lustige Effekte, da beim Speichern ein anderer Style angewendet wird als beim "Clean Up" oder beim "Format Source" ...

Analog sollte man bei der Verwendung von Checkstyle auch in der IDE im Formatter die entsprechenden Checkstyle-Regeln (s.u.) passend einstellen, sonst bekommt man durch Checkstyle Warnungen angezeigt, die man durch ein automatisches Formatieren *nicht* beheben kann.

3.2 Google Java Style und google-java-format

Wer direkt den Google Java Style nutzt, kann auch den dazu passenden Formatter von Google einsetzen: google-java-format. Diesen kann man entweder als Plugin für IntelliJ/Eclipse einsetzen oder als Stand-alone-Tool (Kommandozeile oder Build-Skripte) aufrufen. Wenn man sich noch einen entsprechenden Git-Hook definiert, wird vor jedem Commit der Code entsprechend den Richtlinien formatiert :)

3.3 Spotless und google-java-format in Gradle

Hinweis: Bei Spotless in Gradle müssen je nach den Versionen von Spotless/google-java-format bzw. des JDK noch Optionen in der Datei `gradle.properties` eingestellt werden (siehe Demo und Spotless > google-java-format (Web)).

Tipp: Die Formatierung über die IDE ist angenehm, aber in der Praxis leider oft etwas hakelig: Man muss alle Regeln selbst einstellen (und es gibt *einige* dieser Einstellungen), und gerade IntelliJ "greift" manchmal nicht alle Code-Stellen beim Formatieren. Nutzen Sie Spotless und bauen Sie die Konfiguration in Ihr Build-Skript ein und konfigurieren Sie über den Build-Prozess.

Demo: Konfiguration Formatter (IDE), Spotless/Gradle

4 Metriken: Kennzahlen für verschiedene Aspekte zum Code

Metriken messen verschiedene Aspekte zum Code und liefern eine Zahl zurück. Mit Metriken kann man beispielsweise die Einhaltung der Coding Rules (Formate, ...) prüfen, aber auch die Einhaltung verschiedener Regeln des objektorientierten Programmierens.

4.1 Beispiele für wichtige Metriken (jeweils Max-Werte für PM)

Die folgenden Metriken und deren Maximal-Werte sind gute Erfahrungswerte aus der Praxis und helfen, den Code Smell "Langer Code" (vgl. "Code Smells") zu erkennen und damit zu vermeiden. Über die Metriken *BEC*,

McCabe und *DAC* wird auch die Einhaltung elementarer Programmierregeln gemessen.

- **NCSS** (*Non Commenting Source Statements*)
  - Zeilen pro Methode: 40; pro Klasse: 250; pro Datei: 300
    Annahme: Eine Anweisung je Zeile …
- **Anzahl der Methoden** pro Klasse: 10
- **Parameter** pro Methode: 3
- **BEC** (*Boolean Expression Complexity*)
  Anzahl boolescher Ausdrücke in `if` etc.: 3
- **McCabe** (*Cyclomatic Complexity*)
  - Anzahl der möglichen Verzweigungen (Pfade) pro Methode + 1
  - 1-4 gut, 5-7 noch OK
- **DAC** (*Class Data Abstraction Coupling*)
  - Anzahl der genutzten (instantiierten) "Fremdklassen"
  - Werte kleiner 7 werden i.A. als normal betrachtet

Die obigen Grenzwerte sind typische Standardwerte, die sich in der Praxis allgemein bewährt haben (vergleiche u.a. (Martin 2009) oder auch in AOSP: Write short methods und AOSP: Limit line length).

Dennoch sind das keine absoluten Werte an sich. Ein Übertreten der Grenzen ist ein **Hinweis** darauf, dass **höchstwahrscheinlich** etwas nicht stimmt, muss aber im konkreten Fall hinterfragt und diskutiert und begründet werden!

4.2 Metriken im Beispiel von oben

```
1     private static String lastName;
2     private static MyWuppieStudi studi;
3
4     public static MyWuppieStudi getMyWuppieStudi(String name) {
5         if (studi == null) {
6             studi = new MyWuppieStudi();
7         }
8         if (lastName == null) lastName = name;
9
10        return studi;
11    }
```

- BEC: 1 (nur ein boolescher Ausdruck im `if`)
- McCabe: 3 (es gibt zwei mögliche Verzweigungen in der Methode plus die Methode selbst)
- DAC: 1 (eine "Fremdklasse": `String`)

Anmerkung: In Checkstyle werden für einige häufig verwendete Standard-Klassen Ausnahmen definiert, d.h. `String` würde im obigen Beispiel *nicht* bei DAC mitgezählt/angezeigt.

=> Verweis auf LV Softwareengineering

5 Tool-Support: Checkstyle

Metriken und die Einhaltung von Coding-Conventions werden sinnvollerweise nicht manuell, sondern durch diverse Tools erfasst, etwa im Java-Bereich mit Hilfe von **Checkstyle**.

Das Tool lässt sich Standalone über CLI nutzen oder als Plugin für IDE's (Eclipse oder IntelliJ) einsetzen. Gradle bringt ein eigenes Plugin mit.

- IDE: diverse Plugins: Eclipse-CS, CheckStyle-IDEA
- CLI: `java -jar checkstyle-10.2-all.jar -c google_checks.xml *.java`
- Plugin "**checkstyle**" in Gradle:

```
plugins {
    id "java"
    id "checkstyle"
}

checkstyle {
    configFile file('checkstyle.xml')
    toolVersion '10.2'
}
```

  - Aufruf: Prüfen mit `./gradlew checkstyleMain` (Teil von `./gradlew check`)
  - Konfiguration: `<projectDir>/config/checkstyle/checkstyle.xml` (Default) bzw. mit der obigen Konfiguration direkt im Projektordner
  - Report: `<projectDir>/build/reports/checkstyle/main.html`

Demo: IntelliJ, Checkstyle/Gradle

6 Checkstyle: Konfiguration

Die auszuführenden Checks lassen sich über eine XML-Datei konfigurieren. In Eclipse-CS kann man die Konfiguration auch in einer GUI bearbeiten.

Das Checkstyle-Projekt stellt eine passende Konfiguration für den Google Java Style bereit. Diese ist auch in den entsprechenden Plugins oft bereits enthalten und kann direkt ausgewählt oder als Startpunkt für eigene Konfigurationen genutzt werden.

Der Startpunkt für die Konfigurationsdatei ist immer das Modul "Checker". Darin können sich "FileSetChecks" (Module, die auf einer Menge von Dateien Checks ausführen), "Filters" (Module, die Events bei der Prüfung von Regeln filtern) und "AuditListeners" (Module, die akzeptierte Events in einen Report überführen) befinden. Der "TreeWalker" ist mit der wichtigste Vertreter der FileSetChecks-Module und transformiert die zu prüfenden Java-Sourcen in einen *Abstract Syntax Tree*, also eine Baumstruktur, die dem jeweiligen Code unter der Java-Grammatik entspricht. Darauf können dann wiederum die meisten Low-Level-Module arbeiten.

Eine Reihe von Standard-Checks sind bereits in Checkstyle implementiert und benötigen keine weitere externe Abhängigkeiten. Man kann aber zusätzliche Regeln aus anderen Projekten beziehen (etwa via Gradle/Maven)

oder sich eigene zusätzliche Regeln in Java schreiben. Die einzelnen Checks werden in der Regel als “Modul” dem “TreeWalker” hinzugefügt und über die jeweiligen Properties näher konfiguriert.

Sie finden in der Doku zu jedem Check das entsprechende Modul, das Eltern-Modul (also wo müssen Sie das Modul im XML-Baum einfügen) und auch die möglichen Properties und deren Default-Einstellungen.

```
<module name="Checker">
    <module name="LineLength">
        <property name="max" value="100"/>
    </module>

    <module name="TreeWalker">
        <module name="AvoidStarImport"/>
        <module name="MethodCount">
            <property name="maxPublic" value="10"/>
            <property name="maxTotal" value="40"/>
        </module>
    </module>
</module>
```

Alternativen/Ergänzungen: beispielsweise MetricsReloaded.

Demo: Konfiguration mit Eclipse-CS, Hinweis auf Formatter

7 SpotBugs: Finde Anti-Pattern und potentielle Bugs (Linter)

- **SpotBugs** sucht nach über 400 potentiellen Bugs im Code
  - Anti-Pattern (schlechte Praxis, “dodgy” Code)
  - Sicherheitsprobleme
  - Korrektheit
- CLI: `java -jar spotbugs.jar options ...`
- IDE: IntelliJ SpotBugs plugin, SpotBugs Eclipse plugin
- Gradle: SpotBugs Gradle Plugin

```
plugins {
    id "java"
    id "com.github.spotbugs" version "5.0.6"
}
spotbugs {
    ignoreFailures = true
    showStackTraces = false
}
```

Prüfen mit `./gradlew spotbugsMain` (in `./gradlew check`)

Demo: SpotBugs/Gradle

8 Konfiguration für das PM-Praktikum (Format, Metriken, Checkstyle, SpotBugs)

Im PM-Praktikum beachten wir die obigen Coding Conventions und Metriken mit den dort definierten Grenzwerten. Diese sind bereits in der bereit gestellten Minimal-Konfiguration für Checkstyle (s.u.) konfiguriert.

8.1 Formatierung

- Google Java Style/AOSP: **Spotless**

Zusätzlich wenden wir den Google Java Style an. Statt der dort vorgeschriebenen Einrückung mit 2 Leerzeichen (und 4+ Leerzeichen bei Zeilenumbruch in einem Statement) können Sie auch mit 4 Leerzeichen einrücken (8 Leerzeichen bei Zeilenumbruch) (AOSP). Halten Sie sich in Ihrem Team an eine einheitliche Einrückung (Google Java Style *oder* AOSP).

Formatieren Sie Ihren Code vor den Commits mit **Spotless** (über Gradle) oder stellen Sie den Formatter Ihrer IDE entsprechend ein.

8.2 Checkstyle

- Minimal-Konfiguration für **Checkstyle** (Coding Conventions, Metriken)

Nutzen Sie die folgende **Minimal-Konfiguration** für **Checkstyle** für Ihre Praktikumsaufgaben. Diese beinhaltet die Prüfung der wichtigsten Formate nach Google Java Style/AOSP sowie der obigen Metriken. Halten Sie diese Regeln ein.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE module PUBLIC "-//Checkstyle//DTD Checkstyle Configuration 1.3//EN" "
    https://checkstyle.org/dtds/configuration_1_3.dtd">

<module name="Checker">
  <property name="severity" value="warning"/>

  <module name="TreeWalker">
    <module name="JavaNCSS">
      <property name="methodMaximum" value="40"/>
      <property name="classMaximum" value="250"/>
      <property name="fileMaximum" value="300"/>
    </module>
    <module name="BooleanExpressionComplexity"/>
    <module name="CyclomaticComplexity">
      <property name="max" value="7"/>
    </module>
    <module name="ClassDataAbstractionCoupling">
      <property name="max" value="6"/>
    </module>
    <module name="MethodCount">
      <property name="maxTotal" value="10"/>
```

```
22        <property name="maxPrivate" value="10"/>
23        <property name="maxPackage" value="10"/>
24        <property name="maxProtected" value="10"/>
25        <property name="maxPublic" value="10"/>
26      </module>
27      <module name="ParameterNumber">
28        <property name="max" value="3"/>
29      </module>
30      <module name="MethodLength">
31        <property name="max" value="40"/>
32      </module>
33      <module name="Indentation">
34        <property name="basicOffset" value="4"/>
35        <property name="lineWrappingIndentation" value="8"/>
36        <property name="caseIndent" value="4"/>
37        <property name="throwsIndent" value="4"/>
38        <property name="arrayInitIndent" value="4"/>
39      </module>
40      <module name="TypeName"/>
41      <module name="MethodName"/>
42      <module name="MemberName"/>
43      <module name="ParameterName"/>
44      <module name="ConstantName"/>
45      <module name="OneStatementPerLine"/>
46      <module name="MultipleVariableDeclarations"/>
47      <module name="MissingOverride"/>
48      <module name="MissingJavadocMethod"/>
49      <module name="AvoidStarImport"/>
50    </module>
51
52    <module name="LineLength">
53      <property name="max" value="100"/>
54    </module>
55    <module name="FileTabCharacter">
56      <property name="eachLine" value="true"/>
57    </module>
58    <module name="NewlineAtEndOfFile"/>
59  </module>
```

Sie können diese Basis-Einstellungen auch aus dem Programmiermethoden-CampusMinden/PM-Lecture-Repo direkt herunterladen: checkstyle.xml.

Sie können zusätzlich gern noch die weiteren (und strengeren) Regeln aus der vom Checkstyle-Projekt bereitgestellten Konfigurationsdatei für den Google Java Style nutzen. *Hinweis*: Einige der dort konfigurierten Checkstyle-Regeln gehen allerdings über den Google Java Style hinaus.

8.3 Linter: SpotBugs

- Vermeiden von Anti-Pattern mit **SpotBugs**

Setzen Sie zusätzlich **SpotBugs** mit ein. Ihre Lösungen dürfen keine Warnungen oder Fehler beinhalten, die SpotBugs melden würde.

9 Wrap-Up

- Code entsteht nicht zum Selbstzweck => Regeln nötig!
  - Coding Conventions
    * Regeln zu Schreibweisen und Layout
    * Leerzeichen, Einrückung, Klammern
    * Zeilenlänge, Umbrüche
    * Kommentare
  - Formatieren mit **Spotless**
  - Prinzipien des objektorientierten Programmierens (vgl. "Code Smells")
    * Jede Klasse ist für genau **einen** Aspekt des Systems verantwortlich. (*Single Responsibility*)
    * Keine Code-Duplizierung! (*DRY* - Don't repeat yourself)
    * Klassen und Methoden sollten sich erwartungsgemäß verhalten
    * Kapselung: Möglichst wenig öffentlich zugänglich machen

- Metriken: Einhaltung von Regeln in Zahlen ausdrücken
- Prüfung manuell durch Code Reviews oder durch Tools wie **Checkstyle** oder **SpotBugs**
- Definition des "PM-Styles" (siehe Folie "Konfiguration für das PM-Praktikum")

10 Zum Nachlesen

- Martin (2009)
- Inden (2013, Kap. 13)
- Google Open Source (2022)

Lernziele

- k2: Erklären verschiedener Coding Conventions
- k2: Erklären wichtiger Grundregeln des objektorientierten Programmierens
- k2: Erklären der Metriken NCSS, McCabe, BEC, DAC
- k3: Einhalten der wichtigsten Grundregeln des objektorientierten Programmierens
- k3: Einhalten der wichtigsten Coding Conventions (Formatierung, Namen, Metriken)
- k3: Nutzung des Tools Spotless (Formatierung des Codes)
- k3: Nutzung des Tools Checkstyle (Coding Conventions und Metriken)
- k3: Nutzung des Tools SpotBugs (Vermeiden von Anti-Pattern)

Quellen

Google Open Source. 2022. „Google Java Style Guide“. 2022. https://google.github.io/styleguide/javaguide.html.
Inden, M. 2013. *Der Weg zum Java-Profi*. 2. Aufl. dpunkt.verlag.
Martin, R. 2009. *Clean Code*. mitp.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 71232c0 (tooling: shift headings (use h1 as top-level headings), 2025-04-29)